

3.1__Contadores e Acumuladores

June 8, 2026

0.1 Laços com Contadores, Acumuladores e Maior/Menor

Dentro do contexto de estruturas de repetição (como `for` e `while`), é muito comum utilizarmos variáveis auxiliares para armazenar e atualizar informações ao longo das iterações do laço.

Entre os padrões mais utilizados estão:

- variáveis acumuladoras;
- variáveis contadoras;
- variáveis de maior e menor valor.

0.1.1 Variável Acumuladora

Uma **variável acumuladora** é utilizada para somar ou agregar valores sucessivos durante a execução do laço.

Normalmente, ela é inicializada com o valor 0 antes do início do laço. Essa é uma prática muito comum, pois o zero representa o valor inicial da soma. Durante cada repetição, novos valores são adicionados ao acumulador.

Exemplos de uso:

- calcular a soma de vários números;
- calcular a soma das notas de alunos;
- acumular valores de compras;
- calcular médias.

0.1.2 Variável Contadora

Uma **variável contadora** é utilizada para contar quantas vezes algo acontece durante a execução do laço.

Assim como o acumulador, ela normalmente também começa com o valor 0, pois inicialmente nenhum elemento foi contabilizado.

A variável é incrementada geralmente de 1 em 1 sempre que determinada condição acontece.

Exemplos de uso:

- contar quantos números são pares;
- contar quantos alunos foram aprovados;
- contar quantas vezes um valor aparece;
- contabilizar repetições.

0.1.3 Variáveis de Maior e Menor Valor

Outro padrão muito comum em estruturas de repetição é o uso de variáveis para descobrir o maior ou o menor valor dentro de um conjunto de dados.

Durante o laço, os valores lidos são comparados com os valores atualmente armazenados nas variáveis de maior e menor. Caso um novo valor maior ou menor seja encontrado, a variável é atualizada.

Existem duas abordagens bastante utilizadas:

- inicializar as variáveis com valores muito pequenos ou muito grandes;
- inicializar as variáveis com `None`.

A abordagem com `None` costuma ser mais flexível e segura, pois evita depender de valores arbitrários escolhidos pelo programador.

Esses padrões são fundamentais em lógica de programação e aparecem frequentemente em problemas envolvendo análise de dados, estatísticas e processamento de informações dentro de estruturas de repetição.

0.1.4 Questão 1 - Soma de números digitados

Abaixo temos a questão 1 da lista de exercícios no qual é necessário somar os números digitados.

Para acumular os números digitados foi necessário fazer:

- Uma variável acumuladora (`somaNumeros`)
- Essa variável foi inicializada com zero antes do início do loop
- Depois essa variável é atualizada a cada iteração no loop
- Ao final, ou seja quando sair do loop, você tem nessa variável o somatório

```
[12]: somaNumeros = 0
      for i in range(3):
          numero = int(input("Digite um numero:"))
          somaNumeros = somaNumeros + numero
          #print("iteração:", i, " somaNumeros:", somaNumeros)

      print("\nSoma dos numeros:", somaNumeros)
```

Digite um numero: 3

iteração: 0 somaNumeros: 3

Digite um numero: 2

iteração: 1 somaNumeros: 5

Digite um numero: 5

iteração: 2 somaNumeros: 10

Soma dos numeros: 10

0.2 Biblioteca de Geração de Números Randômicos(aleatórios)

Quando estamos desenvolvendo programas que exigem a entrada de muitos números, digitar manualmente cada valor pode ser cansativo, demorado e sujeito a erros.

A biblioteca `random`, disponível nativamente no Python, resolve esse problema de forma prática e eficiente. Com ela, podemos gerar números aleatórios automaticamente, criando conjuntos de dados de qualquer tamanho com apenas uma linha de código. Isso é especialmente útil em fases de teste e desenvolvimento, onde precisamos de dados variados para verificar se o programa está funcionando corretamente.

Por exemplo, com `random.randint(1, 100)` podemos gerar um número inteiro aleatório entre 1 e 100. Já a função `random.uniform(a, b)` gera números reais entre `a` e `b`.

Além de poupar tempo, o uso de `random` torna os testes mais realistas e abrangentes, já que permite trabalhar com entradas variadas, simulando diferentes cenários de forma rápida.

```
[19]: import random
somaNumeros = 0
for i in range(10):
    numero = random.randint(1, 100)
    somaNumeros = somaNumeros + numero
    print("iteração:", i, " Numero gerado:", numero, " somaNumeros:",
↪somaNumeros)

print("\nSoma dos numeros:", somaNumeros)
```

```
iteração: 0 Numero gerado: 7 somaNumeros: 7
iteração: 1 Numero gerado: 7 somaNumeros: 14
iteração: 2 Numero gerado: 70 somaNumeros: 84
iteração: 3 Numero gerado: 89 somaNumeros: 173
iteração: 4 Numero gerado: 24 somaNumeros: 197
iteração: 5 Numero gerado: 52 somaNumeros: 249
iteração: 6 Numero gerado: 15 somaNumeros: 264
iteração: 7 Numero gerado: 70 somaNumeros: 334
iteração: 8 Numero gerado: 62 somaNumeros: 396
iteração: 9 Numero gerado: 40 somaNumeros: 436
```

Soma dos numeros: 436

```
[25]: import random
somaNumeros = 0
for i in range(10):
    numero = random.uniform(1, 100)
    somaNumeros = somaNumeros + numero
    #print("iteração:", i, " Numero gerado:", numero, " somaNumeros:",
↪somaNumeros)

print("\nSoma dos numeros:", somaNumeros)
```

Soma dos numeros: 524.5847383102956

0.2.1 Questão 2 - Conta Número Pares

```
[24]: import random

contPares = 0
for i in range(10):
    #numero = int(input("Digite um numero:"))
    numero = random.randint(1, 300)
    print(numero)
    if numero %2 == 0:
        contPares +=1 #equivalente -> contPares = contPares + 1

print("Quant. numeros pares:", contPares)
```

```
96
273
11
9
26
207
233
260
161
298
Quant. numeros pares: 4
```

0.2.2 Questão 3 - Calcula Média das idades

```
[ ]: somaIdades = 0
contIdades = 0
for i in range(5):
    idade = int(input("Digite a idade:"))
    somaIdades += idade
    contIdades += 1

media = somaIdades/contIdades
print("A média das idades é:", round(media, 2))
```

0.2.3 Questão 4 - Fatorial

```
[28]: numero = int(input("Digite um numero"))
acumulador = 1
for i in range(1, numero+1):
    acumulador = acumulador * i
```

```
print(f"Fatorial de {numero}!, é {acumulador}")
```

Digite um numero 5

Fatorial de 5!, é 120

0.2.4 Questão 5 - Descobrir o Maior e o Menor Número

Em alguns problemas, precisamos descobrir o maior ou o menor valor dentro de um conjunto de números lidos em um laço de repetição.

Existem diferentes formas de resolver esse problema. Abaixo veremos duas abordagens muito comuns.

Abordagem 1 — Inicializar com um valor muito pequeno ou muito grande Nesta abordagem, iniciamos as variáveis com valores extremos.

- Para descobrir o **maior número**, começamos com um valor muito pequeno.
- Para descobrir o **menor número**, começamos com um valor muito grande.

Assim, qualquer número digitado substituirá o valor inicial na primeira comparação.

```
[1]: import random

maiorNumero = -999999
menorNumero = 999999

for i in range(10):
    numero = random.randint(1, 100)
    print(numero)

    if numero > maiorNumero:
        maiorNumero = numero

    if numero < menorNumero:
        menorNumero = numero

print("\nMaior número:", maiorNumero)
print("Menor número:", menorNumero)
```

59

79

63

83

23

42

27

43

7

94

Maior número: 94

Menor número: 7

Abordagem 2 — Inicializar com None Outra abordagem bastante utilizada é iniciar as variáveis com None.

Nesse caso, verificamos primeiro se a variável ainda não possui valor.

Quando isso acontece, o primeiro número lido passa a ser o maior e o menor ao mesmo tempo.

Essa abordagem é mais flexível, pois evita escolher valores arbitrários muito grandes ou muito pequenos.

```
[2]: import random

maiorNumero = None
menorNumero = None

for i in range(10):
    numero = random.randint(1, 100)
    print(numero)

    if maiorNumero == None or numero > maiorNumero:
        maiorNumero = numero

    if menorNumero == None or numero < menorNumero:
        menorNumero = numero

print("\nMaior número:", maiorNumero)
print("Menor número:", menorNumero)
```

26

57

80

2

29

74

8

62

17

53

Maior número: 80

Menor número: 2

As duas abordagens funcionam corretamente.

A abordagem com None costuma ser considerada mais segura e reutilizável, principalmente quando não sabemos previamente quais valores podem aparecer durante a execução do programa.